LAB MANUAL

Module 4 – Deep Learning

Disclaimer: The content is curated from online/offline resources and used for educational purpose only

Module 4 – Deep Learning

Lab 1. Implementation of Neural Network concepts

Objective

- Build and train a Feedforward Neural Network (FNN) using TensorFlow/Keras for predicting crop yield.
- Train the neural network on the dataset to learn patterns and relationships between environmental factors and crop yield.

Problem

The objective of this project is to develop an Artificial Neural Network (ANN) model to predict crop yield based on various environmental and agricultural factors. These factors include temperature, humidity, rainfall, soil quality, and fertilizer used. This model can assist in agricultural decision-making, helping farmers to forecast crop yields more accurately, optimize farming practices, and plan better for future harvests.

Solution

we'll go through the following steps:

1. Install Required Libraries
2. Feature Selection
3. Normalize the data
4. Split the data into training and testing sets
5. Build the ANN model
6. Compile the model
7. Train the model
8. Evaluate the model

Procedures

Disclaimer: The content is curated from online/offline resources and used for educational purpose only

1. Install Required Libraries

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

2. Feature Selection

```
# Load dataset (replace with your file path)
df = pd.read_csv("dataset/crop_yield.csv")
df.head()
```

```
# Now you can split features and target
X = df.drop('Yield_tons_per_hectare', axis=1)  # Features
y = df['Yield_tons_per_hectare']
```

```
# basic data exploration
for col in X.select_dtypes(include=["object"]).columns.to_list():
    print(f"Unique value in column - {col} are: {X[col].unique()}")
```

3. Data Preprocessing

```
# use one hot encoding to convert the object type columns to numeric column
X=pd.get_dummies(X)
X.head()
```

```
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

4. Split the data into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

Disclaimer: The content is curated from online/offline resources and used for educational purpose only

5. Build the ANN model

```
model = Sequential()
model.add(Dense(64, input_dim=X_train.shape[1], activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1, activation='linear'))  # For regression output
```

6. Compile the model

```
model.compile(optimizer='adam', loss='mean_squared_error', metrics=["mae", "mse"])
```

7. Train the model

```
model.fit(X_train, y_train, epochs=10)
```

```
Epoch 1/10
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 31s 1ms/step - loss: 0.2537 - mae: 0.4019 - mse: 0.2537
Epoch 2/10
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 30s 1ms/step - loss: 0.2536 - mae: 0.4018 - mse: 0.2536
Epoch 3/10
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 31s 1ms/step - loss: 0.2536 - mae: 0.4018 - mse: 0.2536
Epoch 4/10
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 36s 1ms/step - loss: 0.2533 - mae: 0.4016 - mse: 0.2533
Epoch 5/10
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 47s 2ms/step - loss: 0.2533 - mae: 0.4015 - mse: 0.2533
Epoch 6/10
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 43s 2ms/step - loss: 0.2532 - mae: 0.4014 - mse: 0.2532
Epoch 7/10
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 44s 2ms/step - loss: 0.2532 - mae: 0.4015 - mse: 0.2532
Epoch 8/10
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 45s 2ms/step - loss: 0.2530 - mae: 0.4012 - mse: 0.2530
Epoch 9/10
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 40s 2ms/step - loss: 0.2529 - mae: 0.4012 - mse: 0.2529
Epoch 10/10
25000/25000 ━━━━━━━━━━━━━━━━━━━━ 55s 2ms/step - loss: 0.2528 - mae: 0.4012 - mse: 0.2528

<keras.src.callbacks.history.History at 0x1d9db6a4dd0>
```

8. Evaluate the model

Disclaimer: The content is curated from online/offline resources and used for educational purpose only

```
loss = model.evaluate(X_test, y_test)
print(f"Test Loss: {loss}")
```

```
6250/6250 ━━━━━━━━━━━━━━━━━━━━ 8s 1ms/step - loss: 0.2520 - mae: 0.4005 - mse: 0.2520
Test Loss: [0.25202131271362305, 0.4005182385444641, 0.25202131271362305]
```